🔧 Git — Guía Completa de Comandos para la Industria

> Git es el sistema de control de versiones distribuido más usado en la industria. Permite rastrear cambios en el código, colaborar en equipo y mantener un historial completo del proyecto.

📁 1. CONFIGURACIÓN INICIAL

Antes de usar Git por primera vez, debes configurar tu identidad. Esto aparece en cada commit que hagas.

```bash
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"
git config --global core.editor "code --wait"   # Editor por defecto (VS Code)
git config --global init.defaultBranch main      # Rama principal por defecto
git config --list                                # Ver toda la configuración
```

¿Cuándo se usa? Solo una vez al instalar Git en una máquina nueva, o al configurar un nuevo equipo de trabajo.

🏁 2. INICIAR UN REPOSITORIO

```bash
git init                    # Crear un repo nuevo en la carpeta actual
git init nombre-proyecto    # Crear repo en una nueva carpeta
git clone <url>             # Clonar un repo remoto (GitHub, GitLab, etc.)
git clone <url> mi-carpeta  # Clonar en una carpeta con nombre específico
```

¿Cuándo se usa?

- `git init` → al comenzar un proyecto desde cero en tu máquina.
- `git clone` → al unirte a un proyecto existente o descargar un repo de GitHub/GitLab.

📋 3. ESTADO Y DIAGNÓSTICO

Estos comandos te permiten ver en qué estado está tu repositorio en todo momento.

```bash
```

```
git status              # Ver qué archivos cambiaron, están staged o sin rastre
git status -s           # Versión corta y compacta del status
git log                 # Ver historial de commits (detallado)
git log --oneline       # Historial en una línea por commit (muy usado)
git log --oneline --graph --all   # Ver ramas y merges visualmente en terminal
git diff                # Ver cambios que NO están en stage
git diff --staged       # Ver cambios que SÍ están en stage (listos para commit
git diff rama1 rama2    # Comparar dos ramas
```

¿Cuándo se usa? Constantemente durante el desarrollo. `git status` y `git log --oneline` son los más usados del día a día. Antes de hacer un commit, antes de hacer un push, al revisar el trabajo de otros.

➕ 4. STAGING — PREPARAR CAMBIOS

El área de staging (índice) es donde preparas los cambios antes de confirmarlos con un commit.

bash

```
git add archivo.js          # Agregar un archivo específico al stage
git add .                   # Agregar TODOS los cambios del directorio actual
git add -p                  # Modo interactivo — elegir partes específicas del
git restore --staged <file> # Quitar un archivo del stage (sin perder cambios)
git rm archivo.js           # Eliminar archivo del repo y del sistema de archiv
git rm --cached archivo.js  # Dejar de rastrear el archivo sin borrarlo físicam
```

¿Cuándo se usa? Antes de cada commit. `git add .` es el más común en el día a día. `git add -p` se usa cuando quieres ser preciso y hacer commits pequeños y limpios.

💾 5. COMMITS — GUARDAR CAMBIOS

El commit es la unidad fundamental de trabajo en Git. Representa un "snapshot" del proyecto.

bash

```
git commit -m "mensaje descriptivo"       # Crear un commit con mensaje
git commit -am "mensaje"                  # Add + commit en uno (solo archivos
git commit --amend -m "nuevo mensaje"     # Corregir el último commit (mensaje
git commit --amend --no-edit              # Agregar cambios al último commit si
```

Buenas prácticas para mensajes de commit (Conventional Commits)

```
feat: agregar formulario de login
fix: corregir error en cálculo de impuestos
docs: actualizar README con instrucciones de instalación
style: formatear código con Prettier
refactor: extraer función de validación
test: agregar pruebas unitarias para el módulo de pagos
chore: actualizar dependencias de npm
```

¿Cuándo se usa? Después de completar una unidad lógica de trabajo. En la industria se recomienda hacer commits frecuentes y pequeños. El mensaje debe describir el "qué" y el "por qué", no el "cómo".

🌿 6. RAMAS (BRANCHES)

Las ramas permiten trabajar en features, bugs o experimentos sin afectar el código principal.

bash

```bash
git branch                  # Listar ramas locales
git branch -a               # Listar ramas locales Y remotas
git branch nombre-rama      # Crear nueva rama (sin moverse a ella)
git switch nombre-rama      # Cambiarse a una rama existente (moderno)
git switch -c nombre-rama   # Crear Y cambiarse a nueva rama (el más usado)
git checkout nombre-rama    # Alternativa clásica para cambiarse de rama
git checkout -b nombre-rama # Alternativa clásica para crear y cambiarse
git branch -d nombre-rama   # Eliminar rama local (solo si ya fue mergeada)
git branch -D nombre-rama   # Forzar eliminación de rama local
git branch -m viejo nuevo   # Renombrar una rama
```

¿Cuándo se usa?

- `git switch -c feature/login` → al empezar a trabajar en una nueva funcionalidad.
- `git switch main` → para regresar a la rama principal.
- `git branch -d` → después de que un PR/MR fue aprobado y mergeado.

En la industria se trabaja siempre en ramas separadas, nunca directamente en `main`.

🔀 7. MERGE — UNIR RAMAS

Integra el trabajo de una rama en otra.

```bash
git merge nombre-rama                # Merge normal (crea un commit de merge)
git merge --no-ff nombre-rama        # Forzar commit de merge aunque sea fast-
git merge --squash nombre-rama       # Combinar todos los commits en uno solo
git merge --abort                    # Cancelar un merge con conflictos
```

¿Cuándo se usa? Al integrar una feature terminada a `develop` o `main`. En proyectos con GitHub/GitLab, el merge se hace mediante Pull Requests o Merge Requests desde la interfaz web, no siempre por terminal.

🔁 8. REBASE — REESCRIBIR HISTORIAL

Rebase mueve o "replanta" commits sobre otra rama, creando un historial más limpio y lineal.

```bash
git rebase main                  # Rebasar la rama actual sobre main
git rebase -i HEAD~3             # Rebase interactivo de los últimos 3 commits
git rebase --continue            # Continuar después de resolver conflictos
git rebase --abort               # Cancelar el rebase
```

Rebase interactivo (`-i`) — opciones principales:

- `pick` → mantener el commit tal cual
- `squash` (s) → combinar con el commit anterior
- `reword` (r) → cambiar el mensaje del commit
- `drop` (d) → eliminar el commit
- `fixup` (f) → como squash pero descarta el mensaje

¿Cuándo se usa?

- Para limpiar el historial antes de hacer un Pull Request.
- Para mantener la rama actualizada con `main` sin crear commits de merge innecesarios.
- **Regla de oro:** Nunca hagas rebase en ramas públicas/compartidas; solo en tu rama local.

⚔️ 9. RESOLUCIÓN DE CONFLICTOS

Los conflictos ocurren cuando dos ramas modificaron la misma línea de código.

```bash
git status              # Ver qué archivos tienen conflictos
# Editar los archivos con conflictos manualmente o con tu editor
git add archivo.js      # Marcar conflicto como resuelto
git merge --continue    # Continuar el merge
# O si es rebase:
git rebase --continue
```

Estructura de un conflicto en el archivo:

```
<<<<<<< HEAD
código de tu rama actual
=======
código de la rama que estás mergeando
>>>>>>> feature/nueva-funcionalidad
```

Debes decidir qué código conservar (o combinar ambos), eliminar los marcadores `<<<<`, `====`, `>>>>` y luego hacer `git add`.

🌐 10. REMOTOS — TRABAJAR CON GITHUB/GITLAB

```bash
git remote -v                          # Ver remotos configurados
git remote add origin <url>            # Agregar un remoto (llamado "origin")
git remote remove origin               # Eliminar un remoto
git remote rename origin upstream      # Renombrar un remoto
git fetch origin                       # Descargar cambios del remoto sin aplic
git fetch --all                        # Descargar de todos los remotos
git pull                               # Fetch + merge de la rama actual
git pull --rebase                      # Fetch + rebase (historial más limpio)
git push origin nombre-rama            # Subir una rama al remoto
git push -u origin nombre-rama         # Subir y configurar como rama de seguim
git push --force-with-lease            # Push forzado seguro (usado tras rebase
git push origin --delete nombre-rama   # Eliminar rama en el remoto
```

¿Cuándo se usa?

- `git pull --rebase` al inicio del día para sincronizar con el equipo.
- `git push origin feature/mi-feature` al subir tu trabajo para revisión.
- `git push --force-with-lease` después de un rebase (nunca `--force` en ramas compartidas).

🏷️ 11. TAGS — VERSIONES Y RELEASES

Los tags marcan puntos específicos del historial, generalmente versiones de producción.

```bash
git tag                              # Listar todos los tags
git tag v1.0.0                       # Crear tag ligero
git tag -a v1.0.0 -m "Release 1.0"  # Crear tag anotado (recomendado)
git push origin v1.0.0               # Subir un tag específico al remoto
git push origin --tags               # Subir todos los tags
git tag -d v1.0.0                    # Eliminar tag local
git push origin --delete v1.0.0      # Eliminar tag remoto
```

¿Cuándo se usa? Al lanzar una versión a producción. Se sigue el estándar **SemVer** (Versionado Semántico): `MAJOR.MINOR.PATCH` → `v2.1.3`.

🗄️ 12. STASH — GUARDAR CAMBIOS TEMPORALMENTE

Stash guarda tus cambios sin hacer commit, como un "cajón temporal".

```bash
git stash                    # Guardar cambios actuales en el stash
git stash push -m "mensaje"  # Guardar con mensaje descriptivo
git stash list               # Ver todos los stashes guardados
git stash pop                # Aplicar el stash más reciente y eliminarlo
git stash apply stash@{1}    # Aplicar un stash específico sin eliminarlo
git stash drop stash@{0}     # Eliminar un stash específico
git stash clear              # Eliminar todos los stashes
git stash branch nueva-rama  # Crear una rama a partir del stash
```

¿Cuándo se usa? Cuando estás en medio de un trabajo, necesitas cambiar de rama urgentemente (por ejemplo, un bug crítico en producción), pero no quieres hacer commit aún.

⏪ 13. DESHACER CAMBIOS

```bash
```

```
git restore archivo.js          # Descartar cambios locales de un archivo
git restore .                   # Descartar TODOS los cambios locales
git restore --staged archivo.js # Quitar del stage sin perder cambios
git reset --soft HEAD~1         # Deshacer último commit, mantener cambios en s
git reset --mixed HEAD~1        # Deshacer último commit, mantener cambios sin
git reset --hard HEAD~1         # Deshacer último commit y PERDER los cambios
git revert <hash-del-commit>    # Crear un nuevo commit que revierte uno anteri
```

¿Cuándo se usa?

- `git restore` → para descartar cambios no deseados en archivos locales.
- `git reset --soft` → para rehacer el commit con cambios.
- `git revert` → para deshacer commits ya publicados (en ramas compartidas), ya que no reescribe el historial.

🔍 14. BUSCAR EN EL HISTORIAL

bash

```
git log --author="Juan"                    # Commits de un autor específico
git log --since="2024-01-01"               # Commits desde una fecha
git log --grep="login"                     # Commits cuyo mensaje contenga "lo
git log -- archivo.js                      # Historial de un archivo específic
git blame archivo.js                       # Ver quién escribió cada línea
git show <hash>                            # Ver detalles de un commit específ
git bisect start                           # Iniciar búsqueda binaria de un bu
git bisect bad                             # Marcar commit actual como malo
git bisect good <hash>                     # Marcar un commit anterior como bu
```

¿Cuándo se usa?

- `git blame` → para saber quién modificó una línea (investigación de bugs).
- `git bisect` → técnica avanzada para encontrar en qué commit se introdujo un bug.
- `git log --grep` → para buscar cuándo se implementó cierta funcionalidad.

🎣 15. CHERRY-PICK — TOMAR COMMITS ESPECÍFICOS

bash

```
git cherry-pick <hash>           # Copiar un commit específico a la rama actual
git cherry-pick <hash1> <hash2>  # Copiar múltiples commits
git cherry-pick A..B             # Copiar un rango de commits
git cherry-pick --no-commit <hash>  # Aplicar cambios sin crear commit
```

¿Cuándo se usa? Cuando necesitas llevar un fix específico de una rama a otra sin hacer merge completo. Muy útil para aplicar un hotfix de `main` a una rama de release.

🧹 16. LIMPIEZA

bash

```
git clean -n             # Ver qué archivos no rastreados se eliminarían (simu
git clean -f             # Eliminar archivos no rastreados
git clean -fd            # Eliminar archivos Y directorios no rastreados
git clean -fX            # Eliminar solo archivos ignorados (.gitignore)
git gc                   # Limpiar objetos innecesarios y optimizar el repo
```

📝 17. GITIGNORE

El archivo `.gitignore` le dice a Git qué archivos/carpetas ignorar (no rastrear).

```
# Dependencias
node_modules/
vendor/

# Variables de entorno (¡NUNCA subir esto!)
.env
.env.local

# Build / compilados
dist/
build/
*.class
*.pyc

# Sistema operativo
.DS_Store
Thumbs.db

# IDE / Editor
.vscode/
.idea/
*.swp
```

¿Cuándo se usa? Siempre. Debe crearse al inicio de cada proyecto. Nunca se deben subir al repositorio: credenciales, variables de entorno, archivos compilados o dependencias (node_modules).

🔗 18. FLUJOS DE TRABAJO MÁS USADOS EN LA INDUSTRIA

Git Flow (proyectos con releases programadas)

```
main ──────────────────────────────── (producción)
  └── develop ──────────────────────── (integración)
        ├── feature/login ──────────── (nueva funcionalidad)
        ├── feature/pagos ──────────── (nueva funcionalidad)
        ├── release/1.2 ────────────── (preparar versión)
        └── hotfix/bug-critico ─────── (arreglo urgente)
```

GitHub Flow (proyectos con deploy continuo)

```
main ──────────────────────────────── (producción)
  ├── feature/mi-feature ──────────  → PR → merge a main → deploy
  ├── fix/bug-en-login ────────────  → PR → merge a main → deploy
```

📊 RESUMEN RÁPIDO — Comandos del Día a Día

Momento	Comando
Empezar el día	`git pull --rebase`
Nueva tarea	`git switch -c feature/nombre`
Ver qué cambié	`git status` y `git diff`
Preparar commit	`git add .` o `git add -p`
Guardar trabajo	`git commit -m "feat: descripción"`
Subir al remoto	`git push origin feature/nombre`
Limpiar historial	`git rebase -i HEAD~n`
Emergencia	`git stash` → cambiar rama → `git stash pop`
Ver historial	`git log --oneline --graph --all`